Babet Manual

Contents

English | Français

Babet — User Manual

> *Babet*, n.m. — regional word from south-eastern France (Lyonnais, Forez, Dauphiné, Savoie, and adjacent Swiss Romandy) for a pine cone. Small, light, packed with seeds, and good for starting a fire — like this binary.

Babet is a standalone Lua 5.5 binary for Linux scripting and automation. This is the reference manual, organised one file per module so each section stays focused and easy to maintain.

Getting started

- `getting-started.md` — install, first script, the three launch modes (folder / embedded / via PATH).
- `security.md` — what the hardening does and doesn't protect against, recommended least-privilege patterns.

Modules

Each module lives in its own file under `modules/`:

Module	Purpose
`argparse`	Command-line argument parser.
`exec`	Run external programs and capture output.
`fs`	File system : list, copy, hash, attrs.
`http`	HTTP client (built on cpp-httplib).
`inotify`	Filesystem event watching (`inotify(7)`).
`json`	JSON encode/decode (nlohmann/json).
`logging`	Leveled logger.
`signal`	POSIX signals : SIGTERM, SIGINT, …
`socket`	TCP client/server with timeouts.
`sqlite`	Embedded SQL database.
`strings`	String manipulation (`split`, etc.).
`sys`	`which`, `hostname`, `env`, `pid`, …
`tables`	Table manipulation (`mergeTables`, `deepCopyTable`).
`time`	Monotonic and realtime clocks.
`tls`	TLS sockets : `connect_tls`, `starttls`.
`toml`	TOML parser (tomlplusplus).
`user`	System user lookups via NSS.
`workers`	Parallel jobs (real OS threads).

The `user` module is the canonical reference for the documentation pattern : *Why*, *API*, *Quick example*, *Error contract*, *Design decisions*, *Not in v1*. Other modules vary in how strictly they follow that template ; the goal over time is to align them.

Cookbook

`cookbook.md` — concrete recipes : directory watch + email, parallel HTTP fetches, graceful shutdown, TOML+SQLite, etc.

Building a PDF

The manual can be exported to a single PDF :

```
cd docs
./build_doc.sh          # uses pandoc + LaTeX
```

See docs/manual_order_en.txt for the file order ; that's the only place to edit when adding or reordering chapters.

Conventions used in the docs

- **API tables** : every module starts with a small table listing functions, signatures, and what they return.
- **Error contract** : a dedicated section explains which errors are raised (`luaL_error`) versus returned as `(nil, err)`. This is consistent across all modules.
- **Design decisions** : when a choice is non-obvious ("why not recursive ?", "why mandatory ?"), there's a short rationale.
- **Not in v1** : a short list of what could be added additively later without breaking SemVer.

Development methodology

Babet is developed by a single author with substantial AI assistance (primarily Claude / Anthropic, with cross-audits from ChatGPT, Gemini, and Mistral). Every suggestion from these tools is treated as a hypothesis to verify — AI cross-audits regularly identify "bugs" that don't actually exist, or propose API changes for APIs that haven't changed. Every change is validated against the 890-test harness before release.

> **English** | Français

Getting started

Download

Prebuilt binaries for x86_64, aarch64 (RPi4), and armv6l (RPi0) are available on the releases page.

Build from source

Babet vendors all its dependencies (Lua, OpenSSL, SQLite, miniz, nlohmann/json, cpp-httplib, tomlplusplus), so the only prerequisites on your system are :

- a C++23 compiler (recent GCC or Clang)
- CMake (3.20)
- `wget` and `unzip`

```
git clone https://github.com/Chipsterjulien/babet.git
cd babet
./build_local.sh         # downloads deps and compiles
                         # (~5 min on first run, faster afterwards)
./run_tests.sh           # offline harness — should print 827 PASS / 0 FAIL
```

The build script downloads each dependency from its upstream source, verifies the SHA256, then compiles statically. If the upstream is temporarily unreachable (`lua.org` notably has had outages), the script falls back to the Internet Archive's Wayback Machine — the SHA256 check still applies.

The resulting binary is at `test/babet`.

Three ways to run a Lua script

Babet supports three launch modes :

1. Folder mode

Point the binary at a directory containing `main.lua` :

```
echo 'print("hello from Babet")' > main.lua
./test/babet .
```

`require("X")` in `main.lua` looks for `X.lua` in the same directory.

2. Embedded mode (single-file executable)

Package a script and its modules into a self-contained binary :

```
./test/babet --create-exe . myapp
./myapp        # runs main.lua from the embedded ZIP
```

Internally, Babet appends a ZIP to its own binary, and reads `main.lua` (plus any `require`d module) from that ZIP at runtime. The resulting `myapp` is fully self-contained — no Babet install needed on the target machine, just glibc.

3. Embedded via PATH (folder + auto-detect)

If Babet is on `$PATH` and you `chmod +x main.lua` after adding a shebang line :

```
#!/usr/bin/env babet
print("hello")
```

```
chmod +x main.lua
./main.lua
```

Babet uses the script's own directory as the folder, so `require("helpers")` finds `helpers.lua` next to `main.lua`.

Command-line invocation

In addition to the three launch modes above, Babet accepts a few standard flags :

Flag	Effect
`-h`, `--help`	Print the help text and exit `0`.
`-V`, `--version`	Print `babet <version>` and exit `0`.
`-c <dir> <out>`, `--create-exe <dir> <out>`	Create a self-contained executable named `<out>` by embedding `<dir>` (which must contain `main.lua`).

Any other argument starting with `-` is treated as an **unknown option** : Babet prints `Unknown option: ...` + a hint to use `--help`, and exits `1` instead of trying to interpret it as a directory name. Folders whose name legitimately starts with `-` can still be passed via `./-dirname` (POSIX convention).

```
babet --version    # babet 1.7.1
babet --help       # full usage
babet --bogus      # Unknown option: --bogus
                      # Try 'babet --help' for more information.
```

The same version is also exposed to scripts as `babet.VERSION` (plus `babet.VERSION_MAJOR` / `VERSION_MINOR` / `VERSION_PATCH` as integers) — see `sys`.

First script

Once the binary is built, try this :

```
-- main.lua
print("Babet says hi")
print("PID:", babet.pid())
print("Host:", babet.hostname())

local r, err = babet.http.get("https://example.com/")
if r then
    print("Status:", r.status)
else
```

```
    print("HTTP failed:", err)
end
```

Run it with `./test/babet .` (or whichever mode you prefer).

Where to look next

- Each module has its own page under `modules/`.
- The `user` module is a small, complete example of the documentation pattern used throughout — read it as the canonical reference.
- For date and duration utilities (ISO 8601 timestamps, human durations like `"5m"` or `"2h30m"`), see `time` — the `babet.time.*` sub-table covers parsing and formatting.
- See `security.md` before exposing Babet scripts to anything that might receive untrusted input.

  English | Français

Security model

Babet runs trusted Lua scripts. It is not a sandbox.

Scripts execute with the full Lua standard library (`luaL_openlibs`), and Babet additionally exposes filesystem and process primitives such as `exec`, `remove`, `rmdirAll` and `chdir`. A script therefore has the same privileges as the process running it : it can read, modify or delete files and run arbitrary commands. This is by design — like `make`, a shell script, or any build tool, Babet is meant to run code you control.

Only run `main.lua` and `require`d modules that you trust. Do **not** use Babet to execute Lua from untrusted sources ; it provides no isolation against hostile code, and none is intended. If you need to run untrusted scripts, use a purpose-built Lua sandbox instead.

What the hardening *does* protect against

The hardening work in Babet protects *legitimate* use against accidents and supply-chain tampering :

- **Path / symlink handling** uses `lexically_relative()` and component-wise `is_within()` guards (never string prefix checks), so a `cp src dst/` cannot escape into a sibling tree via a symlinked path component.
- **Process-group cleanup** in `babet.exec` ensures that forked children don't outlive the parent or leak zombie processes when the script is interrupted.
- **Output limits** on `babet.exec` bound the amount of stdout and stderr captured into memory (default 16 MiB each, configurable), so a runaway subprocess can't OOM the Babet process.
- **Dependency checksums** : every vendored dependency is SHA256-pinned in `build_local.sh`. An empty hash refuses to build. A mismatch deletes the downloaded file and exits with an error. This protects against a compromised upstream or Wayback Machine archive (which is the fallback source).
- **TLS verification** is on by default (`verify=true`). Disabling it requires an explicit `verify=false` per call — no silent fallback. Custom CA stores can be passed via `ca_cert` or `ca_path`, or via `SSL_CERT_FILE` / `SSL_CERT_DIR` environment variables.

What it does *not* protect against

- A malicious script. Babet has no sandbox. If you `exec` user input, you have a shell injection. If you `loadstring` user input, you have arbitrary code execution.
- A determined attacker who has obtained code execution on the machine running Babet. The hardening makes accidental mistakes loud, not adversarial attacks impossible.
- Long-tail OS-level issues (kernel exploits, container escapes, privilege escalation). Babet is an ordinary userland binary.

Recommended pattern : least privilege

When running Babet as a service, treat it like any other unprivileged process :

```
# As root, create a dedicated service user with no shell.
useradd --system \
        --home-dir /var/lib/myapp \
        --create-home \
        --shell /usr/sbin/nologin \
        --comment "myapp Babet service" \
        myapp

# Use babet.user.exists("myapp") in your install script to
# verify the user is provisioned before launching the daemon.
```

Combine with `systemd` unit file options like `User=myapp`, `PrivateTmp=yes`, `ProtectSystem=strict`, `NoNewPrivileges=yes`, and `CapabilityBoundingSet=` (empty unless you genuinely need a capability). The kernel does much more than Babet can to contain a misbehaving script ; let it.

English | Français

`argparse` — command-line argument parser (Lua module)

A declarative argument parser for command-line scripts. Bundled as a pure-Lua module, loaded via `require("argparse")`. Returns results as data (not via prints or `os.exit`) — the script decides what to do.

Why

A Babet script that takes command-line arguments shouldn't have to reinvent argument parsing (positional vs flag, defaults, help text, type coercion, choices) every time. `argparse` makes the common CLI patterns one-liners — and stays out of the script's way by returning everything as data : help is a result, not a side effect ; errors are a return value, not an exception.

API

```lua
local argparse = require("argparse")
local p = argparse(prog, description?)
```

Builder method	Returns	Use
`p:flag(spec, opts?)`	`p` (chainable)	Boolean flag (no value).
`p:option(spec, opts?)`	`p` (chainable)	Flag that takes a value.
`p:argument(name, opts?)`	`p` (chainable)	Positional argument.
`p:parse(src?)`	`(res, err)`	Parse arguments (see below).
`p:get_usage()`	`string`	Render the help text manually.

`spec` for `flag` and `option` is a **single string** containing one or more names separated by spaces : `"-v"`, `"--verbose"`, or `"-v --verbose"`. Names must start with `-`.

`name` for `argument` is a plain identifier (no leading `-`).

`opts` table fields

Field	Type	Effect
`help`	string	Help text shown by `get_usage()`.
`default`	any	Value returned when the flag/option/argument is absent.

Field	Type	Effect
`dest`	string	Force the destination key in `res`. Default: first long name without dashes, else first short name without dash.
`required`	boolean	Mark as required. For positionals, defaults to `true` unless `default` is set.
`choices`	array of strings	Restrict the raw value to a finite set.
`convert`	function `string -> any`	Coerce the raw string. Returning `nil` is treated as a failure (the convert function may return `(nil, "reason")`).

`parse(src?)` return contract

- **Success** : `(res, nil)` — `res` is a table indexed by `dest`, containing the parsed value (or default) for each declared flag / option / argument.
- **Help requested** : `({ help = true, usage = "<text>" }, nil)` when the user passed `-h` or `--help`. Help is a **success**, not an error. The script decides what to do (typically print `res.usage` and return).
- **Failure** : `(nil, err)` — human-readable string for any user input problem (unknown option, missing value, invalid choice, conversion error, missing required argument, extra positional).

`parse()` **never raises** on user input — every user error goes through `(nil, err)`. Only **builder misuse** (empty spec, name without `-`, duplicate name, etc.) raises via `error()`, because that's a programmer bug.

Source of arguments

- `p:parse()` (no argument) reads the global `arg` table, indices `1..n` (stopping at the first hole). Indices `<= 0` (binary path, folder) are ignored — these are Babet's internal slots, not user arguments.
- `p:parse({ "a", "b" })` parses an explicit array. Useful for testing or for parsing arguments that don't come from the shell.

Accepted argument forms

- `--long val`
- `--long=val`
- `-s val`
- `-s=val`
- `--` ends option parsing — every subsequent token is treated as a positional, even if it starts with `-`.

Short options collapsed together (`-abc`) and the `-fvalue` form (short option immediately followed by its value, no separator) are **not supported** in v1.

Quick example

```lua
local argparse = require("argparse")

local p = argparse("myapp", "Process some files.")
    :flag("-v --verbose", { help = "Verbose output" })
    :option("-o --output",
            { help = "Output file", default = "out.txt" })
    :option("-n --count",
            { help = "How many", convert = tonumber, default = 1 })
    :argument("input", { help = "Input file" })

local args, err = p:parse()
```

```lua
-- Help is a success, not an error.
if args and args.help then
    print(args.usage)
    return
end

if err then
    io.stderr:write("error: ", err, "\n")
    io.stderr:write(p:get_usage(), "\n")
    os.exit(2)
end

print(args.input, args.output, args.count, args.verbose)
```

Running it :

```
$ ./myapp.lua data.csv --count 5 -v
data.csv    out.txt    5    true

$ ./myapp.lua --help
Usage: myapp [options] <input>

Process some files.

Arguments:
  input  Input file

Options:
  -h, --help        show this help
  -v, --verbose     Verbose output
  -o, --output <value>  Output file
  -n, --count <value>   How many
```

Error contract

- **Builder misuse** (`p:flag("")`, `p:option("foo")` without `-`, duplicate name, `p:argument("-bad")`) → raises via `error()`. These are programmer bugs.
- **`p:parse(...)`** never raises on user input. All user input problems are reported as `(nil, "message")` :
  - `"unknown option '--xxx'"`
  - `"option '--xxx' requires a value"`
  - `"flag '-v' does not take a value"`
  - `"missing required option '--xxx'"`
  - `"missing required argument 'name'"`
  - `"argument 'name': invalid choice 'xxx'"`
  - `"option '--xxx': <converter message>"`
  - `"unexpected argument 'xxx'"`
- A `convert` function that itself raises is **caught** by `parse()` and turned into a `(nil, "<...>: conversion error")`. The script never sees an exception coming out of `parse()`.

Design decisions

- **Help is a success, not a side effect**. Returning `{ help = true, usage = "..." }` instead of printing and exiting lets the script decide : print to stdout, log to a file, send to a GUI, prepend a banner, whatever. The library doesn't decide for you, consistent with the rest of Babet where no module unilaterally exits the process.
- **`(res, err)` everywhere on user input, `error()` only on builder bugs**. Same convention as the rest of Babet (`user`, `json`, `sqlite`, etc.).

- **Spec is a single string** (`"-v --verbose"`), not a chain of calls. Discoverable, terse, and avoids ambiguity when multiple names are declared together.
- **`choices` is checked on the raw string, then `convert` applied**. If you pair them, express `choices` as strings — e.g. `{ choices = {"1","2","3"}, convert = tonumber }`. Documented trap, but conscious : doing the reverse would force `choices` values to be of the converted type, which couples two independent features.
- **Positional with `default` is implicitly optional**. Saves the `required = false` boilerplate in the common case.
- **`parse()` reads `arg[1..n]`** — never `arg[0]` or negative indices, where Babet puts its own slots. Just what the user actually typed.

Not in v1

Additive — could be added later without breaking SemVer :

- Sub-commands (`git commit` / `git push` style).
- Combined short options (`-abc` for `-a -b -c`).
- Short options with attached values (`-fvalue`).
- Variadic positionals (`nargs = "+"` or `"*"`).
- Argument groups (visual grouping in help).

  English | Français

`babet.exec` — run an external program

Spawn a subprocess, capture its stdout / stderr, and get its exit code back as structured data. Where `os.execute` gives you just an integer, `babet.exec` gives you a result table with everything you actually need.

Why

Almost every non-trivial script eventually shells out — to `git`, `useradd`, `convert`, `ffmpeg`, whatever. `os.execute` is too crude (no captured output, shell quoting hazards), and `io.popen` doesn't let you both write to stdin and read stdout cleanly, doesn't expose the exit code without parsing, and has no timeout.

`babet.exec` is the structured alternative : argv passed as a list (no shell, no quoting bugs), output captured into memory with a bounded cap, optional timeout, separate stdout/stderr.

API

```
result, err = babet.exec(cmd, args?, opts?)
```

Argument	Type	Notes
`cmd`	string	Program to run. PATH is searched.
`args`	table (optional)	Array of string arguments — each becomes a separate argv element, no shell parsing.
`opts`	table (optional)	See below.

`opts` fields :

Field	Type	Default	Notes
`cwd`	string	inherit	Working directory for the child.
`env`	table `{KEY = value, ...}`	inherit	**Merged** with the parent environment, not replacing it. To unset a variable, omit it from `env`.

Field	Type	Default	Notes
`stdin`	string	`""`	Data piped to the child's stdin.
`timeout`	number (s)	none	Kill the child with SIGKILL if it takes longer.
`max_output`	integer (bytes)	16 MiB	Hard cap **per stream** on captured output. Excess is silently dropped and signalled via `*_truncated`.

Result table on successful launch :

```
{
    stdout = "...",           -- captured stdout, up to max_output bytes
    stderr = "...",           -- captured stderr, up to max_output bytes
    code = 0,                 -- exit code (0..255)
    timed_out = false,        -- true if killed by the timeout
    stdout_truncated = false, -- true if more than max_output was produced
    stderr_truncated = false,
}
```

Quick examples

```lua
-- Basic capture
local r = assert(babet.exec("git", { "rev-parse", "HEAD" }))
if r.code == 0 then
    print("HEAD:", r.stdout:gsub("\n$", ""))
end

-- Non-zero exit is NOT an error
local r = assert(babet.exec("grep", { "needle", "/etc/passwd" }))
if r.code == 0 then
    print("found:", r.stdout)
elseif r.code == 1 then
    print("not found")
else
    print("grep failed:", r.stderr)
end

-- Pipe data to stdin
local r = assert(babet.exec("sha256sum", { "-" }, {
    stdin = "hello world\n",
}))
print(r.stdout)
-- a948904f2f0f479b8f8197694b30184b0d2ed1c1cd2a1ec0fb85d299a192a447  -

-- Augment env (not replace)
local r = assert(babet.exec("env", nil, {
    env = { MY_FLAG = "yes" },
}))
-- Child sees MY_FLAG=yes plus everything the parent already had.

-- Timeout + truncation
local r = assert(babet.exec("yes", nil, {
    timeout = 0.5,
```

```
    max_output = 1024,
}))
print(r.timed_out, r.stdout_truncated)  -- true   true

-- Different cwd
local r = assert(babet.exec("ls", nil, { cwd = "/var/log" }))
```

Error contract

- **`(nil, err)`** only if the launch itself fails : program not found, `cwd` doesn't exist, fork failed, OOM, etc.
- **`(result, nil)`** for every other case, including :
  - The program ran and exited non-zero → `result.code > 0`.
  - The program was killed by the timeout → `result.timed_out` is `true`, `result.code` reflects the signal exit.
  - The program produced more output than `max_output` → the relevant `*_truncated` flag is `true`, captured data stops at the cap.
- **`opts` validation** : invalid types (`opts.cwd` not a string, `opts.env` not a table, `opts.env` keys with `=` or NUL, `opts.timeout` 0 or NaN/Inf, `opts.max_output` non-integer or > 2 GiB) → raise via `luaL_error`. These are caller bugs.

Design decisions

- **Exit code error**. The most common bug with `os.execute` / `exec`-wrappers is treating non-zero exit as an exception. Many programs use exit codes for meaningful state (`grep` returns 1 when no match, `diff` returns 1 when files differ, etc.). The caller decides what counts as failure.
- **No shell**. `args` is an array, each element is one argv slot, no quoting needed and no `$(rm -rf $HOME)` surprises. If you really want a shell, pass `"sh"` + `{ "-c", "your  command" }`.
- **`env` merges with parent**. Replacing the full environment is rarely what you want and usually breaks programs that expect `PATH` or `LANG`. To override one variable, pass it. To unset one, pass it as the empty string and the child will see it empty (POSIX doesn't have "unset" semantics at exec time without extra effort).
- **`max_output` is per stream, not total**. 16 MiB stdout + 16 MiB stderr is the cap. Large enough to capture most build logs, small enough that a runaway subprocess can't OOM the Babet process.
- **Hard 2 GiB cap on `max_output`**. Pretty much anything that needs more than 2 GiB of captured stdout should redirect to a file via `opts.stdin` + a shell wrapper, or via `exec("sh", { "-c", "your-cmd > /tmp/log" })`.
- **`stdin` is a string, not a callback**. Streaming gigabytes into a child is rare and out of scope ; the common case is "feed this small input and read the answer".

Not in v1

- Streaming stdout / stderr (line-by-line callback during execution). Currently the whole capture is returned at once.
- Stdin streaming (callback that produces input chunks).
- Direct fd redirection (`stdout = "/tmp/log"`). Easy to add later if needed ; the shell-wrapper workaround above covers most cases.
- Process group management beyond the child. The current implementation does correctly tear down the child's process group on timeout / interrupt — see `security`.

  English | Français

`babet` fs — file system

A flat set of file system operations exposed directly on `babet` (no sub-namespace, predates the convention). Covers the everyday needs that Lua's `os.*` and `io.*` don't : recursive listing, copy with attributes, hashing, path manipulation, attributes, links.

Why

Lua's standard `io` and `os` give you `open`, `rename`, `remove`, and that's about it. Everything beyond that — listing a directory, recursive copy, computing a hash, querying mtime — requires external programs or low-level FFI. `babet` collects these in a single set of functions that behave consistently (paths are strings, errors come back as `(nil, "...")`).

All paths are accepted as plain strings. Tilde expansion (`~/x`) is **not** automatic — the caller must resolve `$HOME` itself if needed. Functions never expand globs ; for glob support, use `babet.find` with an explicit pattern.

API — Existence, type, attributes

Function	Returns
`babet.fileExists(path)`	`boolean`
`babet.isfile(path)`	`boolean` (true only for regular files)
`babet.isdir(path)`	`boolean`
`babet.fileSize(path)`	`integer` bytes | `(nil, err)`
`babet.getAttributes(path)`	`table` with `mtime`, `mode`, `size`, `uid`, `gid`, `inode`, etc. | `(nil, err)`
`babet.setAttributes(path, uid, gid, mode?)`	`(true, nil)` | `(nil, err)` — set owner/group, optionally mode
`babet.symlinkattr(path)`	same shape as `getAttributes`, but `lstat` (does not follow symlinks) | `(nil, err)`
`babet.getMode(path)`	octal `integer` | `(nil, err)`
`babet.setMode(path, mode)`	`(true, nil)` | `(nil, err)` — `mode` is an integer (use `0x1ed` for `0755`, or build it with `tonumber("755", 8)`)

API — Creating, removing, moving

Function	Returns
`babet.touch(path)`	`(true, nil)` | `(nil, err)` — create file or update mtime
`babet.mkdir(path, opts?)`	`(true, nil)` | `(nil, err)` — `opts.parents = true` for `mkdir -p`
`babet.rmdir(path)`	`(true, nil)` | `(nil, err)` — only empty dirs
`babet.rmdirAll(path)`	`(true, nil)` | `(nil, err)` — recursive
`babet.remove(path)`	`(true, nil)` | `(nil, err)` — file or symlink
`babet.rename(old, new)`	`(true, nil)` | `(nil, err)`
`babet.chdir(path)`	`(true, nil)` | `(nil, err)`
`babet.currentDir()`	`string` (absolute)
`babet.joinPath(a, b, ...)`	`string` — like `path/a/b/c` ; also accepts a single table of segments
`babet.link(target, link, opts?)`	`(true, nil)` | `(nil, err)` — `opts.symbolic = true` for symlinks

API — Path manipulation

These operate on path *strings* — they don't touch the filesystem.

Function	Returns	Example
`babet.getBasename(path)`	`string` | `(nil, err)` — last component	`"/etc/hostname"` → `"hostname"`
`babet.getPath(path)`	`string` | `(nil, err)` — parent dir (dirname)	`"/etc/hostname"` → `"/etc"`

Function	Returns	Example
`babet.getFilename(path)`	`string` | `(nil, err)` — basename without extension	`"report.tar.gz"` → `"report.tar"`
`babet.getExtension(path)`	`string` | `(nil, err)` — final extension	`"report.tar.gz"` → `".gz"`

API — Listing and finding

Function	Returns
`babet.listFiles(path)`	`table` (regular files in `path`) | `(nil, err)`
`babet.find(path, opts)`	iterator function, see below
`babet.createFileIterator(path)`	iterator function over directory entries

`find` accepts `opts` :

- `pattern = "*.lua"` — shell glob applied to file name only.
- `recursive = true` — descend into sub-directories.
- `type = "f"` / `"d"` — restrict to files or dirs.
- `max_depth = N` — limit recursion depth.

API — Copy

Function	Returns
`babet.copy(src, dst, opts?)`	`(true, nil)` | `(nil, err)`
`babet.copyTree(src, dst, opts?)`	`(true, nil)` | `(nil, err)` — recursive
`babet.moveTree(src, dst)`	`(true, nil)` | `(nil, err)`

`copy` options :

- `preserve = true` — keep mtime, mode, owner where possible.
- `overwrite = true` — replace destination if it exists.

API — Hashes and checksums

Hash digests of file contents. Result is **lowercase hex string** unless noted. Note the `sum` suffix — these are the registered names in `babet.*`.

Function	Algorithm	Notes
`babet.crc32sum(path)`	CRC-32 (IEEE 802.3)	**Not cryptographic**. For integrity / accidental-error detection only.
`babet.md5sum(path)`	MD5	Legacy. Don't use for security.
`babet.sha1sum(path)`	SHA-1	Legacy. Don't use for security.
`babet.sha256sum(path)`	SHA-256	
`babet.sha384sum(path)`	SHA-384	
`babet.sha512sum(path)`	SHA-512	
`babet.sha3_256sum(path)`	SHA3-256	
`babet.sha3_384sum(path)`	SHA3-384	
`babet.sha3_512sum(path)`	SHA3-512	
`babet.blake2b512sum(path)`	BLAKE2b-512	

Each returns `string` (hex) or `(nil, err)`. The file is streamed, not loaded entirely into RAM. Non-regular files (directories, sockets, …) are rejected with (nil, "<algo>: not a regular file").

MD5 and SHA-1 are exposed for compatibility (Git, legacy systems) — don't use them for new cryptographic uses. **CRC-32 is even further from cryptographic**: an attacker can produce collisions trivially. Use it for integrity / change detection, never for authentication.

API — In-memory checksums

Compute a hash of bytes already in memory, without round-tripping through a temporary file. Binary-safe (embedded NULs are allowed).

Function	Returns
`babet.crc32(data)`	`string` — 8-char lowercase hex

Cannot fail at runtime (pure computation). Wrong argument types raise via `luaL_error`. As with `crc32sum`, **not cryptographic**.

Quick examples

```
-- Predicates
if babet.isdir("/etc") and babet.fileExists("/etc/hostname") then
    print("size:", babet.fileSize("/etc/hostname"))
end

-- Path manipulation (pure string ops)
local base = babet.getBasename("/var/log/syslog.1")  -- "syslog.1"
local dir  = babet.getPath("/var/log/syslog.1")      -- "/var/log"
local ext  = babet.getExtension("/var/log/syslog.1") -- ".1"

-- Recursive listing
for entry in babet.find("/var/log",
        { pattern = "*.log", type = "f", recursive = true }) do
    print(entry)
end

-- Copy with attributes
babet.copyTree("./src", "/tmp/backup",
                  { preserve = true, overwrite = true })

-- Hash a release tarball
local sha, err = babet.sha256sum("/tmp/babet-1.7.0.tar.gz")
if sha then print("SHA256:", sha) end

-- Quick integrity check (NOT cryptographic, but fast)
local crc = babet.crc32sum("/tmp/babet-1.7.0.tar.gz")
print("CRC32:", crc)                 -- e.g. "a1b2c3d4"

-- In-memory CRC32 of arbitrary bytes (binary-safe, NULs OK)
print(babet.crc32("abc"))          -- "352441c2"
print(babet.crc32(""))             -- "00000000"

-- Set ownership and permissions
local mode_755 = tonumber("755", 8)  -- octal -> integer
assert(babet.setMode("/srv/myapp/run.sh", mode_755))
-- root only:
-- assert(babet.setAttributes("/srv/myapp", 1000, 1000))
```

Error contract

- All functions follow the `(nil, err)` convention on runtime failure (no such file, permission denied, etc.).
- Wrong argument **types** raise via `luaL_error`.
- **`setAttributes(path, uid, gid)`** rejects negative UIDs/GIDs and values above `uid_t`/`gid_t` max (typically `2^32 - 1`). Without that upper-bound check, a value above the limit would truncate silently — worst case to UID 0 (root). Same logic as the `user` module.
- Path safety : `copy`, `copyTree`, `moveTree` use `lexically_relative()` and component-wise `is_within()` guards to prevent symlink-based escapes out of the destination tree. See `security`.

Design decisions

- **Flat namespace** : `fileExists` instead of `fs.exists`, `getAttributes` instead of `fs.attr.get`. Pure historical reason — these functions predate the per-module convention. Stable now ; renaming would break every script.
- **`get*` / `set*` for path attributes** (`getMode`/`setMode`, `getAttributes`/`setAttributes`) but plain verbs for FS actions (`remove`, `touch`, `mkdir`). The “get/set” pairs exist because both directions are needed ; the others are single-direction operations.
- **Path manipulation is separate from FS access**. `getBasename`, `getPath`, etc. operate on strings only — they don’t stat the path. Use `fileExists` if you need to know whether the path actually exists.
- **Hashes have a `sum` suffix** to match the standard Unix tools (`md5sum`, `sha256sum`, etc.). Discoverable for anyone familiar with the command line.
- **`copy` is single-file, `copyTree` is recursive**. Lua doesn’t have function overloading by type, so splitting them avoids the ambiguity “did `copy('dir', 'dir')` mean recursive or not ?”.

Not in v1

- Glob matching as a standalone function (`babet.glob`). Use `find` with `pattern = "*.lua"` for the common case.
- Async I/O. Use `workers` for parallel file processing instead.
- BLAKE2s (only BLAKE2b-512 is exposed via OpenSSL EVP).

  English | Français

`babet.http` — HTTP client

A simple HTTP/HTTPS client built on cpp-httplib, reusing the vendored OpenSSL. Synchronous, blocking, with timeouts.

Why

Almost every non-trivial script needs to talk to some HTTP API. Without a built-in client, you reach for `curl` via `exec`, with all the quoting and process-spawning overhead. `babet.http` makes a request a one-liner, with proper TLS verification by default.

API

Function	Returns
`babet.http.request(opts)`	`response` (table) | `(nil, err)`
`babet.http.get(url, opts?)`	shortcut for `request{ method="GET", url=url, ... }`
`babet.http.post(url, opts?)`	shortcut for `request{ method="POST", url=url, ... }`
`babet.http.put(url, opts?)`	shortcut for `request{ method="PUT", url=url, ... }`
`babet.http.delete(url, opts?)`	shortcut for `request{ method="DELETE", url=url, ... }`

`opts` table

Field	Type	Default
`url`	string (required for `request`)	—
`method`	string	`"GET"`
`headers`	table of `name = value`	`{}`
`body`	string	`""`
`timeout`	number (seconds)	30
`verify`	boolean (TLS cert verification)	`true`
`ca_cert`	string (path to CA bundle file)	system default
`ca_path`	string (path to CA dir)	system default
`follow_redirects`	boolean	`true`
`max_redirects`	integer	5

`response` table

```
{
    status = 200,
    body = "...",
    headers = {                 -- normalised lowercase keys
        ["content-type"] = "application/json",
        ["content-length"] = "42",
    },
    -- final URL after redirects:
    url = "https://api.example.com/v1/data",
}
```

Quick examples

```
-- Simple GET
local r, err = babet.http.get("https://api.example.com/health")
if r then
    print(r.status, r.body)
end

-- JSON POST with auth header
local r, err = babet.http.post("https://api.example.com/v1/things", {
    headers = {
        ["Content-Type"] = "application/json",
        ["Authorization"] = "Bearer " .. token,
    },
    body = babet.json.encode({ name = "widget", count = 7 }),
    timeout = 10,
})

-- Self-signed cert (dev / private CA)
local r = babet.http.get("https://internal.svc/", {
    ca_cert = "/etc/myapp/internal-ca.crt",
})

-- Disable verification (TESTING ONLY)
local r = babet.http.get("https://expired.badssl.com/",
                         { verify = false })
```

Error contract

- **Wrong argument types** → raises via `luaL_error`.
- **Network errors** (DNS, connect, timeout, TLS) → `(nil, "http: <description>")`.

- **HTTP status errors** are **not** errors — a `404` returns a normal `response` table with `status = 404`. Status semantics are the caller's responsibility.

TLS / trust store

Babet ships its own static OpenSSL, so it doesn't automatically inherit the distro's `ca-certificates` configuration. Two mechanisms ensure `verify=true` works out of the box :

1. The vendored OpenSSL is built with `--openssldir=/etc/ssl`, covering Arch, Debian, Ubuntu, Alpine, Gentoo.
2. At TLS init, Babet probes known CA bundle paths for Fedora/RHEL, OpenSUSE, FreeBSD, NetBSD.

You can override per-call with `ca_cert` / `ca_path`, or globally via `SSL_CERT_FILE` / `SSL_CERT_DIR` environment variables. See `security` and `tls` for details.

Design decisions

- **Synchronous, blocking**. Scripts are usually one-shot request-response affairs ; sync is simpler and sufficient. For many parallel calls, use `workers`.
- **`verify=true` by default**. Disabling TLS verification must be explicit (`verify=false`). No silent downgrade.
- **HTTP status is not an error**. The caller decides whether `404` or `500` is failure ; the network call itself succeeded.
- **Headers are normalised to lowercase keys** in the response, to make case-insensitive lookups work directly (`r.headers["content-type"]`).
- **Follow redirects by default**. Bounded to 5 hops, override with `max_redirects` or disable with `follow_redirects=false`.

Not in v1

- Streaming response body (e.g. for downloading large files). Currently `body` is read fully into memory.
- HTTP/2 / HTTP/3.
- WebSockets (use `socket` + TLS + a Lua framing library if needed).
- Server side. cpp-httplib supports it, but exposing a robust HTTP server to Lua is its own design effort.

  English | Français

`babet.inotify` — filesystem event watching

Wraps Linux `inotify(7)` for instant filesystem event delivery — no polling. Used for hot-reloading configuration, watching drop directories, log tailing, file synchronisation triggers, etc.

Why

Polling a directory with `listDir` every second is wasteful (CPU + inode lookups) and laggy. `inotify` is the kernel's purpose-built mechanism : the kernel wakes your script only when something actually changes, with sub-millisecond latency.

The Lua API exposes a minimal contract over a notoriously tricky syscall. Queue overflow is surfaced explicitly (the only thing worse than dropping events silently is not telling you about it).

API

Function	Returns
`babet.inotify.new()`	`watcher` (userdata) | `(nil, err)`
`w:add(path, events, opts?)`	`integer` wd (watch descriptor) | `(nil, err)`
`w:remove(wd)`	`(true, nil)` | `(nil, err)`
`w:read(timeout?)`	`table` of events | `(nil, "timeout")` | `(nil, "interrupted")` | `(nil, err)`
`w:close()`	`(true, nil)` — idempotent

Function	Returns

`events` argument

A strict 1..N array of event name strings. Acceptable names :

Name	Triggered when
`"access"`	file accessed
`"modify"`	file content modified
`"attrib"`	metadata changed (perms, mtime, …)
`"close_write"`	file opened for writing is closed
`"close_nowrite"`	read-only fd closed
`"open"`	file opened
`"moved_from"`	file moved out of watched dir
`"moved_to"`	file moved into watched dir
`"create"`	file/dir created in watched dir
`"delete"`	file/dir deleted from watched dir
`"delete_self"`	the watched path itself is deleted
`"move_self"`	the watched path is moved

Event returned by `read`

```
{
    wd     = 1,                  -- watch descriptor
    name   = "newfile.txt",      -- "" if the watched path itself is the target
    events = { create = true, close_write = true },
    is_dir = false,
    cookie = 0,                  -- non-zero pairs moved_from / moved_to
}
```

Special synthetic event for **queue overflow** :

```
{ wd = -1, events = { overflow = true } }
```

When you see this, the kernel queue ran out of space and some events were lost. Re-scan the watched paths to recover state.

Quick example

```
local w = assert(babet.inotify.new())
assert(w:add("/srv/incoming", { "close_write", "moved_to" }))

babet.signal.handle("TERM", function() w:close(); os.exit(0) end)

while true do
    local events, err = w:read()      -- blocks until something happens
    if not events then
        if err == "interrupted" then break end
        io.stderr:write("inotify: ", err, "\n"); break
    end
    for _, ev in ipairs(events) do
        if ev.events.overflow then
            rescan()                      -- queue overflowed, recover
        else
            handle_new_file("/srv/incoming/" .. ev.name)
        end
```

```lua
    end
end
```

Error contract

- **`read(t)`** where `t` is NaN/Inf/negative → raises via `luaL_error`.
- **`add(path, events)`** with a non-list `events` table (sparse, extra keys, non-string elements) → `(nil, err)`.
- **`add` on a non-existent path** → `(nil, "no such file or directory")`.
- **`read`** :
  - `events` table on success.
  - `(nil, "timeout")` if timeout elapsed.
  - `(nil, "interrupted")` if a handled signal arrived.
  - `(nil, err)` on other errors.
- **Methods after `close`** → `(nil, "inotify: closed")`.

Design decisions

- **Non-recursive in v1**. `inotify(7)` itself isn't recursive ; watching a tree means walking it and calling `add` on each sub-directory, then handling `create` events to extend the watch. That's a real design effort with subtleties (races between walking and event delivery, overflow handling) and worth a dedicated module if/when needed. Buildable in pure Lua on top.
- **Events list mandatory, no implicit "all"**. Watching everything floods the queue and forces every event to traverse Lua. Forcing the caller to pick events makes the cost visible.
- **Overflow surfaced explicitly**, never swallowed. The kernel has a finite queue (`/proc/sys/fs/inotify/max_queued_events`, default 16384). When it overflows, events are lost. The only appropriate action is to re-scan ; silently dropping the signal would defeat the entire reliability story.
- **`read()` is interruptible by signal**. A `read()` blocking forever ignoring `SIGTERM` is the classic graceful-shutdown bug. See [`signal`].
- **`IN_CLOEXEC`** on the fd : not inherited by `exec`'d subprocesses, consistent with sockets.

Not in v1

- Recursive watching. Add at the script level (walk + add) or wait for a `recursive = true` option.
- Multiple watchers per process. Doable today by creating multiple `inotify.new()` instances ; not a built-in registry.
- `IN_MASK_ADD` semantics (adding events to an existing watch). Currently `add(path, events)` replaces the mask. Rare need.

  English | Français

`babet.json` — JSON encode/decode

Backed by nlohmann/json, the most widely-used C++ JSON library. Handles the usual round-trip plus the explicit "empty array" sentinel that Lua tables can't disambiguate from "empty object".

Why

The Lua/JSON impedance mismatch always trips someone up : Lua has no distinction between an empty list and an empty table. Most ad-hoc Lua JSON libraries pick one and get it wrong half the time. `babet.json` is explicit : `{}` decodes to `{}` and re-encodes to `{}` (object), but if you want `[]` you use the `babet.json.empty_array` sentinel.

API

Function	Returns
`babet.json.encode(value, opts?)`	`string` (JSON text) | `(nil, err)`
`babet.json.decode(text)`	`value` | `(nil, err)`

Function	Returns
`babet.json.empty_array`	sentinel value that encodes to `[]`

Encoding options (`opts` table) :

- `pretty = true` — pretty-print with indentation (default `false`).
- `indent = N` — indent width when `pretty=true` (default `2`).

Quick example

```lua
local J = babet.json

-- Decode
local data, err = J.decode([[
{ "name": "alice", "tags": ["admin", "ops"], "active": true }
]])
print(data.name, data.tags[1])

-- Encode (compact)
local out = J.encode({ a = 1, b = { 2, 3 } })
-- out == '{"a":1,"b":[2,3]}'

-- Encode pretty
local pretty = J.encode({ a = 1 }, { pretty = true, indent = 4 })

-- Empty array vs empty object
J.encode({})                    --> "{}"
J.encode(J.empty_array)         --> "[]"
J.encode({ items = J.empty_array })  --> '{"items":[]}'
```

Error contract

- **`encode`** : `(nil, err)` on encoding errors (cycle in the graph, value that doesn't map to JSON like a function or userdata, NaN/Inf numbers, etc.).
- **`decode`** : `(nil, err)` on parse error. The error message includes the line/column where parsing failed.
- **Wrong argument types** → raises via `luaL_error`.

Design decisions

- **`empty_array` sentinel** instead of guessing from the table structure (some libraries treat `{1, 2, 3}` as array and `{a=1}` as object, which is correct, but `{}` is ambiguous). An explicit sentinel removes all surprise.
- **NaN/Inf rejected**, not silently encoded as `null`. Different consumers handle JSON `null` differently for numeric fields ; better to fail loudly and let the caller decide.
- **No streaming API**. Round-trips are bounded by your script's memory regardless ; if you need to handle JSON larger than RAM, reach for a real streaming parser in a separate process.

Not in v1

- Streaming encoder/decoder.
- A schema validator (use a pure-Lua one — they're trivial to add at the script level).

English | Français

`logging` — leveled logger (Lua module)

A standard leveled logger : `debug`, `info`, `warn`, `error`, `fatal`. Bundled as a pure-Lua module, loaded via `require("logging")`. Not in the `babet` namespace — it's a library module, like `inspect`.

Why

Every non-trivial script ends up reinventing some flavour of "levels + timestamps + optional file output". Standardising it removes the bikeshed and makes log output consistent across Babet scripts.

API

```lua
local log = require("logging")
```

Function	Returns
`log.set_level(name)`	`(true, nil)` — `name` is one of `"debug"`, `"info"`, `"warn"`, `"error"`, `"fatal"`
`log.set_output(path)`	`(true, nil)` | `(nil, err)` — file path (default stderr)
`log.debug(...)`, `log.info(...)`, `log.warn(...)`, `log.error(...)`, `log.fatal(...)`	each prints if its level is current threshold

Messages are timestamped (`YYYY-MM-DD HH:MM:SS`) and tagged with the level (`[DEBUG]`, `[INFO]`, etc.). Multiple arguments are concatenated with spaces, like `print`.

Quick example

```lua
local log = require("logging")
log.set_level("info")   -- debug calls below this become no-ops

log.info("startup, pid =", babet.pid())
log.warn("retry attempt", 3, "of", 5)
log.error("connection failed:", err)

-- Optional: route to a file
local ok, err = log.set_output("/var/log/myapp.log")
if not ok then
    log.fatal("can't open log:", err)
    os.exit(1)
end
```

Output :

```
2026-06-11 14:32:01 [INFO] startup, pid = 12345
2026-06-11 14:32:05 [WARN] retry attempt 3 of 5
2026-06-11 14:32:05 [ERROR] connection failed: timeout
```

Error contract

- **`set_level(name)`** : unknown name raises via `error()`.
- **`set_output(path)`** : `(nil, err)` if the file can't be opened for append. Logging continues to the previous sink (stderr if none was set).
- Logging functions themselves never fail — if the file becomes unwritable mid-run, the message is silently dropped (the alternative — crashing the script because the log volume is full — is worse).

Design decisions

- **Pure-Lua module**. No C++ needed, and being in Lua means scripts can monkey-patch it (e.g. add JSON-format output) at the call site without rebuilding Babet.
- **Five levels, no `trace`**. Five is enough for almost everyone ; adding more invites everyone to invent their own.
- **Single global sink**. Multiple loggers / hierarchical loggers / per-module levels are a feature creep trap. If you need multiple destinations, write a wrapper.

Not in v1

- JSON / structured logging output. Easy to bolt on at the script level if needed.
- Log rotation. Use `logrotate(8)` on the file output.
- Async / buffered output. Premature optimisation for typical use.

  English | Français

`babet.signal` — POSIX signals

Register Lua callbacks for POSIX signals (`SIGTERM`, `SIGINT`, `SIGHUP`, …) so long-running scripts can shut down gracefully or reload configuration on demand.

Why

Long-running Babet scripts (bots, daemons, watchers) need to handle signals properly :

- `SIGTERM` from `systemctl stop` should trigger a clean shutdown.
- `SIGINT` from Ctrl-C should do the same.
- `SIGHUP` is the conventional “reload config” signal.

Without explicit handlers, these signals just kill the process, leaving open files, half-written databases, and orphaned children.

API

Function	Returns
`babet.signal.handle(name, fn)`	`(true, nil)` | `(nil, err)` — install Lua callback
`babet.signal.ignore(name)`	`(true, nil)` | `(nil, err)` — set to `SIG_IGN`
`babet.signal.default(name)`	`(true, nil)` | `(nil, err)` — back to OS default
`babet.signal.kill(pid, name)`	`(true, nil)` | `(nil, err)` — send signal to PID
`babet.signal.list()`	`table` of all supported signal names
`babet.signal.is_pending()`	`boolean` — any handled signal queued ?

Signal names accepted (string, case-sensitive) :

`"HUP"`, `"INT"`, `"QUIT"`, `"USR1"`, `"USR2"`, `"PIPE"`, `"ALRM"`, `"TERM"`, `"CHLD"`, `"CONT"`, `"TSTP"`, `"TTIN"`, `"TTOU"`, `"WINCH"`. Other signals (`KILL`, `STOP`) cannot be caught — POSIX forbids it.

Quick example

```lua
local sig = babet.signal
local running = true

sig.handle("TERM", function()
    print("got SIGTERM, shutting down")
    running = false
end)
```

```lua
sig.handle("INT", function()
    print("got SIGINT, shutting down")
    running = false
end)
sig.handle("HUP", function()
    print("got SIGHUP, reloading config")
    cfg = load_config()
end)

while running do
    local line, err = sock:recv_line(1.0)
    if not line then
        if err == "interrupted" then
            -- A handled signal arrived during recv ; the callback
            -- already fired. Re-check the loop condition.
        else
            break
        end
    else
        process(line)
    end
end

print("clean exit")
```

How callbacks run

When a signal arrives :

1. The kernel sets a pending flag (no Lua code runs from the signal handler itself — async-signal-safe restrictions).
2. If the script is in a blocking call (`recv`, `sleep`, `inotify.read`, `accept`, …), the call returns `(nil, "interrupted")`.
3. Before returning, Babet dispatches all pending callbacks in registration order, in the main Lua thread.
4. The script continues normally — the callback can have modified globals (`running = false` is the typical pattern).

This means callbacks always run in Lua context, never from inside a signal handler. They can use any Lua features (no async-signal- safety restrictions).

Error contract

- **Unknown signal name** → `(nil, "signal: unknown name 'XYZ'")`.
- **Calls forbidden inside a worker thread** → `(nil, "signal: not available in worker threads")`. Signals are process-global ; only the main thread manages them.
- **Wrong argument types** → raises via `luaL_error`.
- **`kill(pid, name)`** for a PID that doesn't exist or isn't yours → `(nil, "kill: …")`.

Design decisions

- **Callbacks run in the Lua main thread, not from the signal handler**. POSIX async-signal-safety rules forbid most useful operations from inside a real handler. By marking the signal as pending and dispatching from the next safe point, callbacks can do anything Lua can do.
- **Blocking calls return `"interrupted"` on a handled signal**. Without this, a `recv_line` waiting on the network would block until timeout regardless of `SIGTERM` arriving. With it, the shutdown can complete in milliseconds.
- **No re-entrance**. While a callback is running, additional signals are queued and dispatched after.
- **Forbidden in worker threads**. Signals are process-wide ; only one thread can sensibly own them. Workers must use channels for inter-thread communication.

Not in v1

- `sigprocmask` / fine-grained signal masking. Not commonly needed at script level.
- `signalfd`-based polling integration. The current dispatch model is simpler and covers the typical cases.

  English | Français

`babet.socket` — TCP client and server

Low-level TCP sockets with timeouts, signal awareness, and a small set of higher-level helpers (`recv_line`, `recv_all`). Foundation for any custom protocol — the IRC bot, a JSON-over-TCP daemon, etc. See `tls` for the encrypted variant.

Why

For everyday HTTP needs there's `http`. But many scripts need to speak a custom line-oriented protocol (IRC, Redis, plain telnet-style daemons), where a real TCP socket with timeouts is the right tool. Lua has no built-in TCP, and `LuaSocket` is a separate dep ; `babet.socket` makes the common patterns one-liners.

API

Constructors

Function	Returns
`babet.socket.connect(host, port, opts?)`	`socket` | `(nil, err)`
`babet.socket.listen(host, port, opts?)`	`server_socket` | `(nil, err)`

`opts` :

- `timeout = N` — default timeout in seconds for blocking I/O (default infinite).
- `connect_timeout = N` — only for `connect`, default 30 s.
- `nodelay = true` — set `TCP_NODELAY`.
- `keepalive = true` — set `SO_KEEPALIVE`.

Methods (client socket)

Method	Returns
`s:send(data)`	`integer` bytes sent | `(nil, err)`
`s:recv(n, timeout?)`	`string` | `(nil, "timeout")` | `(nil, "interrupted")` | `(nil, err)`
`s:recv_line(timeout?)`	`string` (without `\n`) | `(nil, …)`
`s:recv_all(timeout?)`	`string` (read until EOF) | `(nil, …)`
`s:set_timeout(seconds)`	sets default timeout (`0` = infinite)
`s:close()`	idempotent
`s:peer()`	`(host, port)` | `(nil, err)` — remote endpoint

`recv_line` reads bytes until `\n` (LF) is found. CR is stripped if present (`\r\n` → returned without the trailing `\r`). Has a hard 8 MiB cap to prevent DoS via an endless single line.

Methods (server socket)

Method	Returns
`srv:accept(timeout?)`	`socket` | `(nil, …)`

Method	Returns
`srv:close()`	idempotent

Quick examples

TCP echo client

```lua
local s = assert(babet.socket.connect("localhost", 4000, {
    timeout = 5,
    nodelay = true,
}))

s:send("hello\n")
local reply, err = s:recv_line()
print("got:", reply)
s:close()
```

Simple line-protocol server with graceful shutdown

```lua
local sig = babet.signal
local srv = assert(babet.socket.listen("0.0.0.0", 4000))
local running = true
sig.handle("TERM", function() running = false end)
sig.handle("INT", function() running = false end)

while running do
    local client, err = srv:accept(1.0)        -- 1 s timeout
    if not client then
        if err == "timeout" or err == "interrupted" then
            -- loop and re-check `running`
        else
            io.stderr:write("accept: ", err, "\n"); break
        end
    else
        repeat
            local line = client:recv_line(30)
            if line then client:send("you said: " .. line .. "\n") end
        until not line
        client:close()
    end
end
srv:close()
```

Error contract

- **Connect errors** (DNS, refused, timeout) → `(nil, "socket: ...")`.
- **Bind errors** (port in use, permission) → `(nil, "socket: ...")`.
- **`recv`** :
  - `string` (any length up to `n` requested).
  - Empty string `""` when the peer closed cleanly. Not an error.
  - `(nil, "timeout")` if no data within timeout.
  - `(nil, "interrupted")` if a handled signal arrived.
  - `(nil, "line too long")` for `recv_line` past the 8 MiB cap.
  - `(nil, err)` on other errors.
- **`send`** : may return fewer bytes than requested. Wrap in a loop if you need to send a fixed amount (or use `recv_all`- symmetric semantics in your protocol).

- **Methods after `close`** → `(nil, "socket: closed")`.
- **NaN/Inf timeouts**, negative timeouts → raises via `luaL_error`.

Design decisions

- **Single-threaded, blocking I/O**. No `select`/`poll` exposed — use `workers` for concurrent connections, or short timeouts in a polling loop for simple cases.
- **`recv_line` has an 8 MiB cap**. Protects against a peer that sends megabytes without ever sending `\n`. Configurable later if a use case needs more.
- **Signal-aware blocking**. All `recv*` and `accept` return `"interrupted"` on a handled signal, so the script can exit cleanly.
- **`recv` returns `""` on clean close, not `nil`**. Lets scripts distinguish "remote went away" (`""`) from "no data yet" (`"timeout"`) without inventing yet another sentinel.

Not in v1

- UDP sockets. Easy to add additively (`babet.socket.udp.*`).
- Unix-domain sockets. Same reasoning.
- Native multiplexing (`select`/`poll`/`epoll` exposed to Lua). Use workers or short-timeout polling instead.

  English | Français

`babet.sqlite` — embedded SQL database

Wraps SQLite 3.53.1 (vendored), the most-deployed database engine in the world. Zero-config persistence for any script that needs more than a JSON file but less than a database server.

Why

A script that needs to keep state across runs (a bot's seen-list, a scraper's progress, accumulated metrics) shouldn't have to pull in PostgreSQL or invent its own file format. SQLite is exactly right at this scale : single file, transactional, fast, no daemon.

The Lua API mirrors the SQLite C API closely (open / exec / prepare / step / finalize) with safer defaults and Lua-friendly error returns.

API

Opening and closing

Function	Returns
`babet.sqlite.open(path, opts?)`	`db` (userdata) | `(nil, err)`
`db:close()`	`(true, nil)` — idempotent

`opts` :

Field	Type	Default
`readonly`	boolean	`false`
`wal`	boolean — enable WAL journal mode	`false`
`busy_timeout`	number ms (block this long on a locked db)	`5000`
`foreign_keys`	boolean	`true`

Special path `":memory:"` opens an in-memory database (lost on close). Use for tests or transient processing.

Direct exec (no parameters / one-shot statement)

Function	Returns
`db:exec(sql)`	`(true, nil)` | `(nil, err)`
`db:exec(sql, params)`	same, with `?`-bound parameters
`db:query(sql, params?)`	`table` of row tables | `(nil, err)`

Prepared statements (re-use, performance)

Function	Returns
`db:prepare(sql)`	`stmt` (userdata) | `(nil, err)`
`stmt(params?)`	iterator over rows
`stmt:exec(params?)`	`(true, nil)` | `(nil, err)`
`stmt:finalize()`	`(true, nil)` — idempotent

Transactions

Function	Returns
`db:transaction(fn)`	`(true, result)` if `fn` returns truthy ; rollback + `(false, err)` otherwise

Utilities

Function	Returns
`db:last_insert_rowid()`	`integer`
`db:changes()`	`integer` — rows affected by last write
`db:in_transaction()`	`boolean`

Type mapping

SQLite type	Lua type
INTEGER	integer
REAL	number (float)
TEXT	string
BLOB	string (binary-safe)
NULL	`nil` (read), `nil` or `db.NULL` sentinel (write)

Quick examples

```lua
local db = assert(babet.sqlite.open("state.db", { wal = true }))

-- Schema
db:exec([[
    CREATE TABLE IF NOT EXISTS users (
        id INTEGER PRIMARY KEY AUTOINCREMENT,
        name TEXT UNIQUE NOT NULL,
        active INTEGER DEFAULT 1
    );
]])
```

```lua
-- Insert with parameters
assert(db:exec("INSERT INTO users (name) VALUES (?)", { "alice" }))
print("inserted as id", db:last_insert_rowid())

-- Query rows
local rows = assert(db:query("SELECT id, name FROM users WHERE active = ?", { 1 }))
for _, r in ipairs(rows) do
    print(r.id, r.name)
end

-- Prepared statement reused in a loop (fast path)
local stmt = assert(db:prepare("INSERT INTO users (name) VALUES (?)"))
for _, name in ipairs({ "bob", "carol", "dave" }) do
    assert(stmt:exec({ name }))
end
stmt:finalize()

-- Transaction
local ok, err = db:transaction(function()
    db:exec("UPDATE users SET active = 0 WHERE name = ?", { "alice" })
    db:exec("UPDATE users SET active = 1 WHERE name = ?", { "bob" })
    return true   -- commit
end)
if not ok then print("rollback:", err) end

db:close()
```

Error contract

- All runtime errors → `(nil, "sqlite: <description>")` with the SQLite error code in the message.
- **`exec(sql, params)` with `?` placeholders but no `params` argument** → `(nil, "sqlite: placeholders found but no params  table provided")` — explicit error rather than silent NULL binding, which is a classic injection footgun.
- **Sparse keys in `params`** (e.g. `{[1] = "a", [3] = "c"}`) → `(nil, err)`.
- **Empty BLOB string** → stored correctly as empty BLOB (previously buggy in pre-1.5).
- **Methods after `close`** → `(nil, "sqlite: db closed")`.
- **Wrong argument types** → raises via `luaL_error`.

Design decisions

- **WAL is opt-in, not default**. WAL is strictly better for most use cases, but it creates `*-wal` and `*-shm` sidecar files that some users find surprising. Opt-in keeps the default surprise-free, but every long-running daemon should pass `wal=true`.
- **Foreign keys ON by default**, contrary to SQLite default. Most modern schemas rely on FK constraints ; defaulting them off is a footgun.
- **`transaction(fn)` commits if `fn` returns truthy**. Conventional in Lua : return a value to signal success, return nothing/`nil`/`false` (or raise) to rollback. The function's return value is propagated.
- **Placeholders without `params` is an error**, not a silent NULL bind. Catches a class of injection-adjacent bugs early.
- **Errors are returned, not raised**. Even malformed SQL returns `(nil, err)`. Scripts that don't check are noisy but not catastrophic.

Not in v1

- `BLOB` streaming I/O (`sqlite3_blob_open`). Use string serialisation if you fit in memory.

- Virtual tables / FTS5 / R-Tree. Available via raw SQL if the vendored SQLite is built with them ; no Lua-level API yet.
- Backup API (`sqlite3_backup_init`). For now, `VACUUM INTO  'backup.db'` works as a one-liner.

For a higher-level ORM-like layer, build it in Lua on top of this API.

English | Français

`babet` strings — string manipulation

A single function : split a string by a separator into a table. Predates the sub-namespace convention, lives directly on `babet`.

Why

Splitting a string by a delimiter is one of the most common string operations, and Lua's standard library doesn't have one out of the box (`string.gmatch` works but requires pattern escaping). A dedicated function is more discoverable and avoids the pattern-escaping trap.

API

Function	Returns
`babet.split(s, sep)`	`table` (array) of substrings

- `s` : the string to split.
- `sep` : the separator string (literal, no patterns).

If `sep` doesn't appear in `s`, the result is a one-element table containing the whole `s`. If `s` is empty, returns an empty table.

Quick example

```lua
local parts = babet.split("a,b,c,d", ",")
-- parts == { "a", "b", "c", "d" }

local one = babet.split("hello", ",")
-- one == { "hello" }
```

Error contract

- **Wrong argument types** → raises via `luaL_error`.
- Otherwise always succeeds.

Design decisions

- **Literal separator, not Lua pattern**. The common case is splitting CSV-like data, where you want `.` to mean a literal dot, not "any character". If you need pattern matching, fall back to `string.gmatch`.
- **Empty entries are preserved**. `"a,,b"` splits into `{"a", "", "b"}`. Consumers who want to filter empty strings do it in one line of Lua.

Not in v1

- Pattern-based splitting (with escape rules). Use `string.gmatch` if needed.
- `joinTable(t, sep)` mirror — `table.concat` already exists in the stdlib.

English | Français

`babet` sys — system utilities

Process and host introspection helpers : PID, hostname, kernel info, executable lookup, environment variables.

Why

Lua's standard library has `os.getenv` and `os.execute`, but nothing for the PID, hostname, kernel info, or a portable `which`. These are universal scripting needs that don't deserve a roundabout via `popen` calls.

These functions live directly on `babet` (flat, no sub-table) because they predate the per-module convention used by recent additions.

API

Function	Returns
`babet.pid()`	`integer` — current process ID
`babet.hostname()`	`string` | `(nil, err)` — system hostname
`babet.uname()`	`table` with `sysname`, `nodename`, `release`, `version`, `machine`
`babet.which(cmd)`	`string` (absolute path) | `(nil, "not found")`
`babet.env(name)`	`string` | `nil` — like `os.getenv`, but consistent
`babet.setenv(name, value)`	`(true, nil)` | `(nil, err)`
`babet.getMemoryUsage()`	`integer` — Lua VM memory in bytes (after a full GC)
`babet.getDetailedMemoryUsage()`	`(integer, integer)` — currently both equal the GC count in bytes ; kept as two returns for API stability

Runtime constants

The same `babet` table also exposes constants describing the Babet binary that's running the script. Useful for logging, gating features behind a minimum version, or sanity-checking the runtime.

Constant	Type	Example
`babet.VERSION`	`string`	`"1.7.1"`
`babet.VERSION_MAJOR`	`integer`	`1`
`babet.VERSION_MINOR`	`integer`	`7`
`babet.VERSION_PATCH`	`integer`	`1`

The string version is what `babet --version` prints. The integer components are there for programmatic comparison without parsing the string.

```lua
-- Log the runtime
print("running under Babet " .. babet.VERSION)

-- Gate a feature behind a minimum version
local need_minor = 7
if babet.VERSION_MAJOR < 1
   or (babet.VERSION_MAJOR == 1 and babet.VERSION_MINOR < need_minor) then
    error("this script needs Babet >= 1." .. need_minor)
end
```

Quick example

```lua
print("Running on:", babet.hostname(), "PID:", babet.pid())

local u = babet.uname()
print("Kernel:", u.sysname, u.release, u.machine)
```

```lua
-- Find a binary
local sh, err = babet.which("bash")
if sh then
    babet.exec({ sh, "-c", "echo hello" })
end

-- Read/write env
print("HOME:", babet.env("HOME"))
babet.setenv("MY_VAR", "value")

-- Memory introspection (forces a full GC first)
print("Lua VM memory:", babet.getMemoryUsage(), "bytes")
local a, b = babet.getDetailedMemoryUsage()
print("Detailed:", a, b)
```

Error contract

- **`pid()`** : never fails, always returns an integer.
- **`hostname()`** / **`which()`** : `(value, nil)` on success, `(nil, err_string)` on failure.
- **`env(name)`** : `value` if set, `nil` if unset. Never raises.
- **`setenv(name, value)`** : `(true, nil)` on success, `(nil, err)` on failure (rare — usually OOM or invalid name).
- **`uname()`** : never fails on any supported system, always returns the full table.
- **Wrong argument types** (e.g. `which(42)`) → raises via `luaL_error` after string coercion as per Lua convention. Pass a table or boolean to force a real error.

Design decisions

- **`env(name)` returns `nil`, not `""`, for unset variables**. Matches `os.getenv` semantics. Tests should use `env("X") == nil`, not `env("X") == ""`.
- **`which` returns the absolute path**, not just "yes/no". This is more useful : you can `exec` the returned path directly. For a boolean check, use `which(cmd) ~= nil`.
- **`uname()` returns a table, not multiple values**. Makes it forward-compatible if a field is ever added (e.g. `domainname`).

Not in v1

Additive — could be added later :

- `unsetenv` (just `setenv(name, nil)` could work too).
- `getuid` / `getgid` / `getppid` / `getlogin`.
- Full environment dump as a table.

For user lookups (UID → name, etc.) see the dedicated `user` module — that uses NSS, which is the right backend for that question.

English | Français

`babet` tables — table manipulation helpers

Two helpers that fill obvious gaps in the Lua standard library : a proper deep copy, and a multi-source merge.

Why

Lua's standard library has no built-in deep copy and no multi-source table merge. Both are routine needs : merging config from defaults + overrides, snapshotting a table before mutation, copying a tree of options. Writing them per-script is repetitive and error-prone (missing the metatable, cycles, etc.).

These two functions live directly on the `babet` table (no sub-namespace) because they predate the per-module convention used by recent additions.

API

Function	Returns
`babet.mergeTables(t1, t2, ...)`	new `table` — string keys merged (last writer wins), numeric keys appended in order
`babet.deepCopyTable(t)`	new `table`, recursively copied

`mergeTables` is variadic — pass any number of tables. The inputs are not mutated. The merge has two rules :

- **String keys** : later tables overwrite earlier ones at the same key (last writer wins).
- **Numeric keys (array part)** : concatenated in the order tables are passed, then in 1..n order within each table. So `mergeTables({1, 2}, {3, 4})` gives `{1, 2, 3, 4}`, not `{3, 4}`.

`deepCopyTable` copies nested tables recursively. Cycles are detected (no infinite loop). Non-table values (numbers, strings, booleans, functions, userdata) are not duplicated — they're referenced as-is.

Quick example

```lua
-- String keys : last writer wins
local defaults = { port = 8080, host = "localhost", debug = false }
local user_cfg = { port = 9000, debug = true }
local cfg = babet.mergeTables(defaults, user_cfg)
-- cfg.port  == 9000      (user_cfg overrode)
-- cfg.host  == "localhost"
-- cfg.debug == true

-- Numeric keys : append in order
local a = { 1, 2 }
local b = { 3, 4 }
local merged = babet.mergeTables(a, b)
-- merged == { 1, 2, 3, 4 }   (NOT { 3, 4 })

-- Deep copy : snapshot before mutation
local snapshot = babet.deepCopyTable(cfg)
cfg.port = 9999             -- does NOT affect snapshot
print(snapshot.port)        -- still 9000
```

Error contract

- **Wrong argument types** (passing a non-table to either function) → raises via `luaL_error`.
- Neither function returns errors via `(nil, err)`. They either succeed or raise.

Design decisions

- **Shallow values are referenced, not deep-copied**, in `deepCopyTable`. Copying functions or userdata makes no sense, and copying immutable values (numbers, strings, booleans) changes nothing. Only tables are recursively duplicated.
- **Metatables are not copied** in `deepCopyTable`. This is a conscious choice : preserving a metatable across a copy can surprise callers (the copy still triggers `__index` callbacks that mutate the original). If you need to copy with metatable, do it explicitly with `setmetatable(babet.deepCopyTable(t), getmetatable(t))`.
- **`mergeTables` is shallow**. If two source tables share a sub- table at the same string key, the result references the last one unchanged — it does not recurse into it. Combine with `deepCopyTable` if you need merge-then-isolate semantics.

- **Numeric keys are appended, not overwritten**. The choice comes from the most common Lua use case : combining list-shaped tables (`{1, 2}` + `{3, 4}` → `{1, 2, 3, 4}`). If you wanted overwriting semantics on numeric keys, build a destination table and assign explicitly.

Not in v1

Additive — could be added later without breaking SemVer :

- A `deepMergeTables` that recurses into nested tables when both sources have a table at the same key. Common request, but the semantics around lists vs maps gets ambiguous, hence not in v1.
- An `equalsTables` for deep structural comparison. Easy to write in pure Lua, hasn't surfaced as a real need yet.

  English | Français

`babet` time — monotonic and realtime clocks

Two clocks with sub-second precision and a sleep that honours both `s`/`ms`/`us`/`ns` units and signals. Fills the gaps in `os.time` / `os.clock`. Lives directly on `babet`, no sub-namespace.

Why

Lua's `os.time()` only gives seconds, and `os.clock()` measures CPU time, not wall time. Neither is appropriate for measuring real durations (request latency, retry backoff, timeouts). And `os` has no portable nanosecond sleep.

`babet.monotonic()` and `babet.now()` expose `CLOCK_MONOTONIC` (durations) and `CLOCK_REALTIME` (timestamps), with `babet.sleep()` honouring signal interruption.

API

Function	Returns
`babet.monotonic()`	`number` — seconds since arbitrary epoch (immune to clock changes)
`babet.now()`	`number` — POSIX time (seconds since 1970-01-01 UTC)
`babet.sleep(amount, unit?)`	`(true, nil)` | `(nil, "interrupted")`

`unit` for `sleep` is one of `"s"` (default), `"ms"`, `"us"`, `"ns"`. Floats are accepted.

API — date and duration utilities (v1.8.0+)

Added under the `babet.time` sub-table. The three flat functions above are also aliased there (`babet.time.now`, `babet.time.monotonic`, `babet.time.sleep`) for ergonomy.

Function	Returns
`babet.time.iso(ts?)`	`string` — `"YYYY-MM-DDTHH:MM:SSZ"` (UTC). Without arg, formats the current time.
`babet.time.parse_iso(s)`	`(integer, nil)` | `(nil, "parse_iso: ...")` — Unix timestamp in seconds.
`babet.time.parse_duration(s)`	`(integer, nil)` | `(nil, "parse_duration: ...")` — number of seconds.
`babet.time.format_duration(n)`	`string` — `"1d2h3m4s"` style, compact, with zero components omitted.

ISO 8601 format accepted by `parse_iso` — pragmatic, but with one strict rule : **a timezone suffix is required**.

- Date and time separator : `T` or space.
- Timezone : `Z` (UTC), `+HH:MM`, or `-HH:MM`. **No** `+0200`, `+02`, or trailing absence — these are all rejected. Hours `00..23`, minutes `00..59`.
- Fractional seconds (`.123`) are accepted but ignored.
- A string without timezone is rejected explicitly rather than interpreted as UTC, because most real-world timestamps without a `Z` are local, not UTC, and silent misinterpretation is a common bug source.

Duration format accepted by `parse_duration` :

- Units : `s`, `m` (minute), `h`, `d`. No `w`, `mo`, `y`, `ms`, `us`, `ns` in v1.
- Compose unit chunks without whitespace : `"1h30m"`, `"2d12h"`, `"45m30s"`, `"1d1h1m1s"`.
- Units must appear in **strictly decreasing order** (`d > h > m   > s`). `"30m1h"` is rejected.
- No duplicate unit : `"1h1h"` is rejected.
- No sign, no spaces. Empty string is rejected. A bare number without unit (`"5"`) is rejected.
- `"0s"` is the only canonical representation of zero (matches `format_duration(0)`).

Round-trip invariant : for every integer `n >= 0`, `parse_duration(format_duration(n)) == n`. This is checked in the test suite over a representative sample.

Quick example

```lua
-- Measure a duration safely (immune to NTP / DST jumps)
local t0 = babet.monotonic()
do_something()
local elapsed = babet.monotonic() - t0
print(string.format("took %.3f s", elapsed))

-- Timestamp an event
local now = babet.now()
db:exec("INSERT INTO events (ts, kind) VALUES (?, ?)", { now, "ping" })

-- Sleep 100 ms, but wake up on a signal
local ok, err = babet.sleep(100, "ms")
if not ok and err == "interrupted" then
    print("woken by a signal")
end

-- ISO 8601 timestamp for logs (always UTC)
print("event at " .. babet.time.iso())          -- "2026-06-17T14:32:01Z"
print(babet.time.iso(0))                        -- "1970-01-01T00:00:00Z"

-- Parse a log line back to a Unix timestamp
local ts = assert(babet.time.parse_iso("2026-06-17T10:00:00+02:00"))
-- ts is 1750147200 (which is 08:00:00 UTC)

-- Human-readable durations
local cache_ttl = babet.time.parse_duration("2h30m")  -- 9000
print("cache valid for " .. babet.time.format_duration(cache_ttl))
-- "cache valid for 2h30m"
```

Error contract

- **`monotonic()` / `now()`** : never fail. Always return a number.
- **`sleep(amount, unit)`** :
  - `(true, nil)` if the sleep completed normally.
  - `(nil, "interrupted")` if a handled signal arrived during the sleep (see `signal`).
- **NaN / Inf / negative amount** → raises via `luaL_error`.
- **Unknown unit string** → raises via `luaL_error`.

Design decisions

- **Two clocks, two functions, no flag**. `monotonic()` for durations, `now()` for timestamps. Conflating them in one function with a parameter would invite the wrong choice.
- **`now()` for realtime**, not `realtime()` — shorter, and the natural English meaning matches what's expected (current time).
- **Signal-aware `sleep`**. A 60-second sleep that ignores `SIGTERM` is the classic graceful-shutdown bug. `sleep()` returns `(nil, "interrupted")` so the caller can exit cleanly.
- **No "format date" helper**. `os.date(format, babet.now())` works fine, and a duplicate function would just be a wrapper.

Not in v1

- `babet.boottime()` (`CLOCK_BOOTTIME` — includes suspend). Rarely needed, easy to add later.
- `babet.utcnow()` / `localnow()` helpers that return pre-formatted strings. Trivial in Lua, no value-add.

  English | Français

`babet.socket` TLS — `connect_tls` and `starttls`

The TLS half of `socket` : encrypted TCP for IRC over TLS, IMAPS, custom secure protocols. Two entry points : `connect_tls` for protocols that go TLS from the start (port 6697 IRC, 993 IMAPS, etc.) and `starttls` for protocols that upgrade an existing plain connection (SMTP, IMAP, IRC `STARTTLS`).

Why

Talking encrypted TCP without `http` is a real need : IRC bots over `+6697`, SMTP submission, custom internal services behind TLS. Doing it manually with `openssl s_client` via `exec` is unreliable ; `openssl` the C++ library + `cpp-httplib` chosen abstractions give a clean Lua API that handles cert verification correctly by default.

API

Function	Returns
`babet.socket.connect_tls(host, port, opts?)`	`tls_socket` | `(nil, err)`
`s:starttls(opts?)`	`(true, nil)` | `(nil, err)` — upgrade an existing plain socket

`opts` (merged with the usual socket opts) :

Field	Type	Default
`verify`	boolean	`true`
`ca_cert`	string (path)	system bundle
`ca_path`	string (path)	system bundle
`server_name`	string	host argument
`alpn`	array of strings	none

A `tls_socket` has the same methods as a plain socket (`send`, `recv`, `recv_line`, `recv_all`, `set_timeout`, `close`, `peer`). The encryption is transparent.

Quick examples

Connect to IRC over TLS

```
local s = assert(babet.socket.connect_tls("irc.libera.chat", 6697, {
    timeout = 30,
```

```lua
}))

s:send("NICK babet-bot\r\n")
s:send("USER babet 0 * :babet bot\r\n")

while true do
    local line, err = s:recv_line()
    if not line then break end
    print(line)
    if line:match("^PING (.+)$") then
        s:send("PONG " .. line:match("^PING (.+)$") .. "\r\n")
    end
end
```

STARTTLS upgrade (SMTP submission)

```lua
local s = assert(babet.socket.connect("mail.example.com", 587))
print(s:recv_line())                           -- 220 banner
s:send("EHLO myhost\r\n")
repeat
    line = s:recv_line()
until not line:match("^250%-")                 -- last 250 line

s:send("STARTTLS\r\n")
print(s:recv_line())                           -- 220 ready to start TLS

assert(s:starttls({ server_name = "mail.example.com" }))
-- s is now encrypted ; continue with EHLO + AUTH + …
```

Self-signed server (dev / private CA)

```lua
local s = assert(babet.socket.connect_tls("internal.svc", 5555, {
    ca_cert = "/etc/myapp/internal-ca.crt",
}))
```

Skip verification (TESTING ONLY)

```lua
local s = assert(babet.socket.connect_tls("self-signed.local", 8443, {
    verify = false,
}))
```

Error contract

- All errors prefixed `"tls: ..."` (handshake, cert verify, etc.) or `"socket: ..."` (DNS, connect, timeout).
- `(nil, err)` always — no exceptions for "expected" failures like cert mismatch.
- **`verify=true` + invalid cert** → `(nil, "tls: certificate verify failed: <reason>")`. Common reasons : expired, self-signed, hostname mismatch, unknown CA.
- **NaN/Inf timeouts**, wrong types → raises via `luaL_error`.

Trust store

Babet ships its own statically-linked OpenSSL. Out of the box, `verify=true` works on :

- **Arch, Debian, Ubuntu, Alpine, Gentoo** : via `--openssldir=/etc/ssl` baked into the vendored OpenSSL.
- **Fedora, RHEL, CentOS, Rocky, Alma, OpenSUSE, FreeBSD, NetBSD** via runtime probing of known CA bundle paths.

Overrides, in priority order :

1. `ca_cert` / `ca_path` per call.
2. `SSL_CERT_FILE` / `SSL_CERT_DIR` environment variables.
3. The two mechanisms above.

If none of these find a CA store and you use `verify=true`, the handshake fails with a clear error. See `security` for the full picture.

Design decisions

- **TLS 1.2 minimum**. SSL 3, TLS 1.0, TLS 1.1 are unconditionally rejected — they're broken, no modern server should rely on them. TLS 1.3 is negotiated automatically when available.
- **`verify=true` by default, `verify=false` requires explicit opt-in**. No way to accidentally turn off cert checking.
- **`server_name` defaults to the host argument** to make SNI Just Work. Pass it explicitly if you're connecting by IP to a TLS server that uses SNI.
- **Same methods as plain socket**, so a function that takes a `socket` works for both transparently.

Not in v1

- Client certificate authentication. Possible to add as a `client_cert` / `client_key` opt later.
- TLS session resumption / session tickets. Not commonly needed at script level.
- OCSP stapling verification. Reliance on OpenSSL's defaults.

  English | Français

`babet.toml` — TOML parser

Backed by toml++, a strict TOML 1.0 parser. Decode only — encoding TOML faithfully from a Lua table is ambiguous (Lua doesn't distinguish between inline tables and dotted tables, for instance).

Why

TOML is the natural config-file format for modern tools (Cargo, pyproject, systemd-credentials, etc.). A Babet script reading its own config should be able to do it directly, not via a TOML → JSON conversion via `tomlq`.

API

Function	Returns
`babet.toml.decode(text)`	`table` (parsed TOML) | `(nil, err)`

The returned table mirrors the TOML structure :

- Strings → Lua strings (UTF-8)
- Integers → Lua integers
- Floats → Lua numbers
- Booleans → Lua booleans
- Datetimes → Lua strings in ISO 8601 form (TOML has no Lua-native equivalent ; use `os.date` or a date library if you need parsed components)
- Arrays → Lua tables (1-indexed)
- Tables (`[section]` or `{ inline }`) → Lua tables (with string keys)

Quick example

```lua
local body = [[
title = "My App"
[server]
```

```
host = "localhost"
port = 8080
features = ["auth", "metrics"]

[database]
url = "sqlite:///var/lib/app.db"
]]

local cfg, err = babet.toml.decode(body)
if not cfg then error("config parse failed: " .. err) end

print(cfg.title)                  -- "My App"
print(cfg.server.host)            -- "localhost"
print(cfg.server.features[1])     -- "auth"
print(cfg.database.url)           -- "sqlite:///var/lib/app.db"
```

Error contract

- **`decode`** : `(nil, err)` on parse error. The error message includes the line/column where parsing failed, and a short description of what was wrong (per toml++).
- **Non-string argument** → raises via `luaL_error`.

Design decisions

- **Decode only, no encode**. Round-tripping a TOML file via a Lua table loses formatting (comments, dotted vs inline tables, newlines between sections) that real users care about. If you need to write TOML, build the string yourself — TOML is designed to be human-writable. A faithful encoder would be a large undertaking for limited value.
- **Datetimes as strings**. TOML's datetime type is rich (offset, local, date-only, time-only) but Lua has no native equivalent. Returning a string in ISO 8601 form preserves the data losslessly and lets the script parse it with the tools it prefers.

Not in v1

- Encoder. May be added if a real use case appears, but unlikely.
- A schema validator. Build one in Lua if you need it ; the decoded structure is just a table.

  English | Français

`babet.user`

System user lookups via NSS, without parsing `/etc/passwd` directly.

Why

On a modern Linux system, users may come from multiple sources : `/etc/passwd`, LDAP, SSSD, NIS+, FreeIPA, systemd-userdb, or other NSS plugins. Reading `/etc/passwd` directly misses everything except the first source. `babet.user` is backed by `getpwnam_r(3)` and `getpwuid_r(3)`, which traverse the resolver chain configured in `/etc/nsswitch.conf` — exactly what `id` or `getent passwd` does.

API

Function	Returns
`user.get(name_or_uid)`	`table` | `(nil, "user not found")` | `(nil, "user: <err>")`
`user.exists(name_or_uid)`	`boolean`

`name_or_uid` accepts :

- a **string** → looked up by name via `getpwnam_r`
- a **non-negative integer** → looked up by UID via `getpwuid_r`

Any other type, a float, a negative integer, a string containing an embedded NUL byte, or an integer above `uid_t` max raise via `luaL_error` — these are caller bugs, not runtime conditions to handle gracefully.

Returned table on success

```
{
    name  = "yaourt",
    uid   = 968,
    gid   = 968,
    gecos = "yaourt AUR build user",
    home  = "/var/cache/yaourt",
    shell = "/usr/sbin/nologin",
}
```

All string fields are guaranteed non-nil. They may be empty strings when the underlying NSS source has no value (this is rare but possible — typically empty `gecos`).

Quick example

```lua
-- Ensure a service user is provisioned before launching the daemon.
if not babet.user.exists("yaourt") then
    error("yaourt user not provisioned — run useradd first")
end

local u = assert(babet.user.get("yaourt"))
babet.chdir(u.home)
```

Error contract

- **Bad argument type** (`table`, `boolean`, `nil`, float, negative integer, UID above `uid_t` max, string with embedded NUL) → raises via `luaL_error`. Use `pcall` if you need to recover.
- **User absent** (the resolver returned no match) → `get()` returns `(nil, "user not found")` ; `exists()` returns `false`.
- **NSS error** (resolver itself broken : LDAP unreachable, out of memory, etc.) → `get()` returns `(nil, "user: <description>")` ; `exists()` returns `false` (errors are silently coerced into "absent" for prediates ; use `get()` if you need to diagnose).

Design decisions

- **NSS only, never `/etc/passwd`** : reading the file directly would silently miss every account that isn't in the local file. The whole point of `babet.user` is that the script gets the same answer as `id` or `getent passwd`.
- **Thread-safe _r variants** : Babet exposes `babet.workers`, so we use `getpwnam_r`/`getpwuid_r` everywhere.
- **`gecos` exposed raw** : the historical format is `Full Name,Office,WorkPhone,HomePhone[,Other]`, but in practice most entries hold a free-form name. Parsing it in Lua is trivial if you need to ; baking a parser in C++ would be premature.
- **NUL byte rejection** : `"root\0evil"` would be seen as `"root"` by `getpwnam_r` (it stops at the first NUL). On user-derived input, that could bypass an upstream identity check. Explicit rejection is safer than implicit truncation.
- **`uid_t` range check** : on Linux, `uid_t` is `uint32_t`, while `lua_Integer` is `int64_t`. Without a check, `5_000_000_000` would be silently truncated to a 32-bit value and could match an unrelated account.

Not in v1

Additive — these could be added later without breaking SemVer :

- Group lookups (`getgrnam` / `getgrgid`) — will be added when a concrete use case asks for them.
- `/etc/shadow` access (passwords, password aging) — out of scope. Requires root and is rarely useful outside niche admin scripts. Password authentication should go through PAM, not Babet.
- User/group creation — already feasible via `babet.exec("useradd …")` and `groupadd`. No need for a dedicated binding.

  English | Français

`babet.workers` — parallel jobs

Run Lua code on real OS threads. Useful for I/O-bound work that benefits from concurrency (fetching many URLs, scanning many files, processing batches) and CPU-bound work on multi-core hardware.

Why

Lua has coroutines, but those run cooperatively on a single OS thread — fine for state machines, not for using multiple cores. And many tasks are I/O-bound : 100 HTTP requests that each take 500 ms still take 50 s sequentially, but ~500 ms in parallel.

`babet.workers` spawns real OS threads each running their own isolated Lua state, communicating with the parent via passed-in arguments and return values. No shared state means no locks, no data races — just send work in, get results out.

API

Function	Returns
`babet.workers.spawn(code, args?)`	`job` (userdata) | `(nil, err)`
`babet.workers.cpu_count()`	`integer` — number of logical CPUs
`job:join(timeout?)`	`(true, result)` | `(false, err)` | `(nil, "timeout")`
`job:done()`	`boolean` — non-blocking check
`job:cancel()`	`(true, nil)` — cooperative ; sets a flag

`code` argument

A Lua source string that runs in the worker. The full `babet` namespace is available, plus a `worker` global with :

- `worker.args` — the second argument passed to `spawn`.
- `worker.cancelled()` — returns `true` if the parent called `job:cancel()`. The worker is expected to check this in long loops.

The worker's last expression value (or `return value`) is what `join()` returns as `result`.

`args` argument

Any value that can be deep-copied between Lua states : `nil`, booleans, numbers, strings, tables of the same. **Not** : functions, userdata, threads, tables with cycles. Tables are deep-copied — no sharing.

Quick examples

Parallel HTTP fetches

```lua
local urls = {
    "https://a.example/",
    "https://b.example/",
    "https://c.example/",
    "https://d.example/",
}
```

```lua
local jobs = {}
for i, url in ipairs(urls) do
    jobs[i] = babet.workers.spawn([[
        local url = worker.args.url
        local r, err = babet.http.get(url, { timeout = 10 })
        if not r then return { ok = false, err = err } end
        return { ok = true, status = r.status, length = #r.body }
    ]], { url = url })
end

for i, job in ipairs(jobs) do
    local ok, result = job:join()
    if ok then
        print(i, result.status or result.err)
    else
        print(i, "worker crashed:", result)
    end
end
```

CPU-bound work with cancellation

```lua
local n = babet.workers.cpu_count()
print("running on", n, "cores")

local jobs = {}
for i = 1, n do
    jobs[i] = babet.workers.spawn([[
        local chunk = worker.args.chunk
        local total = 0
        for x = chunk.from, chunk.to do
            if x % 1000 == 0 and worker.cancelled() then
                return { cancelled = true, partial = total }
            end
            total = total + heavy_compute(x)
        end
        return { total = total }
    ]], { chunk = { from = i * 1000, to = (i + 1) * 1000 - 1 } })
end

-- ... later, if needed:
-- for _, j in ipairs(jobs) do j:cancel() end
```

Worker pool pattern

```lua
local function map_parallel(items, code, max_concurrent)
    max_concurrent = max_concurrent or babet.workers.cpu_count()
    local results = {}
    local i = 1
    local active = {}

    while i <= #items or next(active) do
        -- launch as many as we can
        while i <= #items and #active < max_concurrent do
            active[#active + 1] = {
                index = i,
                job = babet.workers.spawn(code, { item = items[i] }),
            }
```

```lua
            i = i + 1
        end
        -- harvest any finished
        for k = #active, 1, -1 do
            local entry = active[k]
            if entry.job:done() then
                local ok, result = entry.job:join()
                results[entry.index] = ok and result or { err = result }
                table.remove(active, k)
            end
        end
        if next(active) then babet.sleep(10, "ms") end
    end
    return results
end
```

Error contract

- `spawn` : `(nil, err)` if the OS thread can't be created (rare — usually resource limits).
- `join` :
  - `(true, result)` — worker completed normally ; `result` is its return value (or `nil` if it didn't return).
  - `(false, err)` — worker crashed with an uncaught error ; `err` is the error message + traceback.
  - `(nil, "timeout")` — `timeout` elapsed without the worker finishing.
- `cancel` never fails. The flag is set ; the worker may take a while to notice (it must call `worker.cancelled()`).
- **Wrong argument types** → raises via `luaL_error`.

Sharing data

Workers don't share Lua state. Communication is :

- **In** : via the `args` second argument to `spawn` (deep-copied into the worker's state).
- **Out** : via the worker's return value (deep-copied back).
- **Side effects** : a worker can write to a file, append to a log, query a database — but coordination through such side effects is the script's responsibility.

If two workers write to the same SQLite file, set `wal=true` on `open` so they don't lock each other out. SQLite handles the synchronisation correctly with WAL ; in `busy_timeout` you can configure how long to wait on a busy db.

Design decisions

- **One Lua state per worker, no sharing**. The alternative — shared state with locks — is a well-known source of subtle bugs and we don't think Lua-level locks are worth the design effort at this layer. Pure message-passing is simpler.
- **No global thread pool**. Each `spawn` creates a fresh OS thread, each `join` closes it. For tight loops, build a pool in Lua (see example above). The simpler model wins on legibility.
- **Cooperative cancellation, not preemptive**. There's no way to forcibly stop a Lua thread in the middle of an operation without leaving the runtime in an undefined state. `worker.cancelled()` returns `true` ; the worker is responsible for checking it periodically.
- **`babet.signal` is not available in workers**. Signals are process-wide ; only the main thread can sensibly own them.

Not in v1

- Channels (FIFO queues between workers). Add later if a use case shows the pattern is common.
- Shared memory (mmap). Same.
- Async I/O futures (`spawn().then(...)` style). Out of scope ; use a polling loop or `done()` checks.

  English | Français

Cookbook

Concrete recipes for common scripting tasks. Each recipe is short and self-contained ; copy, adapt, ship.

Watch a directory and email new files

A drop directory that mails out every file as it arrives. Uses `babet.inotify` for instant wakeup (no polling), and listens on `close_write` + `moved_to` so the file is guaranteed complete.

```lua
local w = assert(babet.inotify.new())
assert(w:add("/srv/incoming", { "close_write", "moved_to" }))

babet.signal.handle("TERM", function()
    w:close()
    os.exit(0)
end)

while true do
    local events, err = w:read()
    if not events then
        if err == "interrupted" then break end
        io.stderr:write("inotify: ", err, "\n"); break
    end
    for _, ev in ipairs(events) do
        if ev.events.overflow then
            -- queue overflowed: rescan the directory
            -- (events were lost)
        else
            local path = "/srv/incoming/" .. ev.name
            send_email(path)
            os.remove(path)
        end
    end
end
```

Ensure a service user exists, then drop to it

A typical install-time check before launching a daemon as a non-root user.

```lua
local function ensure_user(name)
    if babet.user.exists(name) then return true end
    local ok, err = babet.exec("useradd", {
        "--system",
        "--home-dir", "/var/lib/" .. name,
        "--create-home",
        "--shell", "/usr/sbin/nologin",
        name,
    })
    if not ok then
        return nil, "useradd failed: " .. tostring(err)
    end
    return true
end

assert(ensure_user("myapp"))
local u = assert(babet.user.get("myapp"))
babet.chdir(u.home)
```

Fetch many URLs in parallel

`babet.workers` runs Lua on multiple cores. Total wall-clock is close to the slowest single fetch, not the sum.

```lua
local urls = { "https://a.example/", "https://b.example/",
               "https://c.example/", "https://d.example/" }

local jobs = {}
for i, url in ipairs(urls) do
    jobs[i] = babet.workers.spawn([[
        local url = worker.args.url
        local r, err = babet.http.get(url, { timeout = 10 })
        if not r then return { error = err } end
        return { status = r.status, length = #r.body }
    ]], { url = url })
end

for i, job in ipairs(jobs) do
    local ok, result = job:join()
    print(i, ok, result and result.status or result.error)
end
```

Graceful shutdown of a long-running script

Catch `SIGTERM` / `SIGINT` so the script can close resources properly. Works with `systemctl stop` and Ctrl-C.

```lua
local log = require("logging")
log.set_level("info")

local running = true
babet.signal.handle("TERM", function()
    log.info("SIGTERM, shutting down")
    running = false
end)
babet.signal.handle("INT", function()
    log.info("Ctrl-C, shutting down")
    running = false
end)

while running do
    -- main loop ; blocking calls return (nil, "interrupted") when
    -- a handled signal arrives, so the loop exits quickly.
    local line, err = sock:recv_line()
    if not line then
        if err == "interrupted" then break end
        log.error("recv: ", err); break
    end
    process(line)
end

sock:close()
log.info("clean shutdown")
```

Read a TOML config, validate, persist to SQLite

```lua
-- Load TOML
local f = assert(io.open("config.toml", "r"))
local body = f:read("a"); f:close()
```

```lua
local cfg, perr = babet.toml.decode(body)
if not cfg then error("config.toml: " .. perr) end

-- Validate (cheap manual checks ; for a real schema, build one in Lua)
assert(type(cfg.server) == "table", "missing [server]")
assert(type(cfg.server.host) == "string", "missing server.host")
assert(math.type(cfg.server.port) == "integer", "server.port must be integer")

-- Open db, persist
local db = assert(babet.sqlite.open("state.db", { wal = true }))
db:exec([[CREATE TABLE IF NOT EXISTS config (k TEXT PRIMARY KEY, v TEXT)]])
db:exec("INSERT OR REPLACE INTO config VALUES (?, ?)",
        { "host", cfg.server.host })
db:exec("INSERT OR REPLACE INTO config VALUES (?, ?)",
        { "port", tostring(cfg.server.port) })
db:close()
```

Cache with a human-readable TTL, expiry as ISO timestamps

Parse a TTL from config (`"15m"`, `"2h30m"`, `"1d"`), compute a deadline, and log it in a uniform UTC format that grep'ing across log files actually works on.

```lua
-- Config from TOML, CLI, env, anywhere.
local raw_ttl = "15m"

local ttl, err = babet.time.parse_duration(raw_ttl)
if not ttl then error("bad TTL '" .. raw_ttl .. "': " .. err) end

local deadline = babet.now() + ttl
print(string.format("[%s] cache entry created, expires at %s (TTL %s)",
    babet.time.iso(),
    babet.time.iso(deadline),
    babet.time.format_duration(ttl)))

-- ...later, somewhere else in the script:
if babet.now() >= deadline then
    print("[" .. babet.time.iso() .. "] cache miss: entry expired")
end
```

Useful properties of this pattern :

- `babet.time.iso()` is always UTC, always 20 characters, always the same shape. `grep` and `sort` work on the timestamps directly.
- `parse_duration("15m")` returns an integer, so `now() + ttl` is exact (no float drift over long durations).
- `format_duration(ttl)` round-trips : `parse_duration(    format_duration(n)) == n` for every `n >= 0`. Safe to use as a canonical representation in logs or databases.

Pretty-print a Lua value

The bundled `inspect` module gives readable output for nested tables.

```lua
local inspect = require("inspect")
print(inspect({ a = 1, b = { c = 2, d = { 3, 4, 5 } } }))
-- { a = 1, b = { c = 2, d = { 3, 4, 5 } } }
```

Hello there

The minimal Babet script :

```lua
babet.helloThere()
```